

Проект «Эмулятор микрокалькулятора «Электроника МК-61s mini»

https://github.com/UN7FGO/MK61S_MINI

TinyBASIC

Версия описания: 06.10.2026

TinyBASIC — отдельный режим МК61s mini со своим меню, типом сохранённых программ и общим с FOCAL построчным редактором. Ядро совместимо с Palo Alto Tiny BASIC v3 Li-Chen Wang (1977): целые номера строк, несколько команд через ;, сокращения с точкой, LET, PRINT, INPUT, IF, GOTO, GOSUB, RETURN, FOR, NEXT, STOP, массив @() и функции ABS, RND, SIZE, COLS, ROWS. Двоеточие : сохранено как однозначный дополнительный разделитель команд.

При этом числовая семантика должна быть калькуляторной, а не 16-битной целочисленной. TinyBASIC для МК61s mini — это не побитовая эмуляция PATB, а компактный BASIC с синтаксисом Tiny BASIC и математикой МК-61.

Основные решения

- У TinyBASIC отдельное меню и отдельный список программ.
- Программа состоит из строк с целыми номерами 1...32767.
- Одна строка может содержать несколько команд, разделённых ; или :.
- Синтаксис программы всегда латинский: операторы, функции и переменные записываются английскими именами даже в русском режиме интерфейса.
- В русском режиме локализуются меню, подсказки, сообщения и ошибки.
- Числа, арифметика, тригонометрия и случайные числа используют калькуляторную семантику МК-61, а не 16-битные целочисленные правила PATB.
- Деление / является обычным вещественным делением: 14/5 дает 2.8.
- Сравнения возвращают 1 или 0.
- IF считает истинным любое ненулевое значение выражения.
- LET можно писать явно, но можно опускать: A=10 эквивалентно LET A=10.
- THEN в IF можно писать явно, но можно опускать.
- Ключевые слова можно сокращать по правилам Tiny BASIC: P. значит PRINT, G. значит GOTO, GOS. значит GOSUB.
- Одно-клавишная вставка операторов, как в FOCAL, в TinyBASIC не используется.
- Редактор TinyBASIC использует общий код редактора, но языковые hooks не вставляют операторы по одной клавише.
- TinyBASIC включён по умолчанию и отключается условной компиляцией через MK61_ENABLE_TINYBASIC.
- Внутренняя сигнатура сохраненной программы: B2.
- Файловое расширение для virtual FAT: TBI.
- Исходником считается обычный текст TinyBASIC. Компилированное представление может быть RAM-кэшем, но не источником истины.

Отличия от исторического Palo Alto Tiny BASIC

Синтаксис и порядок исполнения PATB сохранены, но числовая и аппаратная среда адаптированы к МК61s mini.

Изменено:

- числа не ограничены диапазоном - 32767...32767;
- арифметика не целочисленная;
- / не отбрасывает дробную часть;
- логические операции работают как булевы операции над числами, а не как побитовые операции над 16-битными словами;
- IF проверяет ненулевое значение, а не нечетность;
- RND () вызывает генератор случайных чисел MK61s mini;
- ошибки должны быть понятны пользователю MK61s mini, но короткие английские формы WHAT?, HOW?, SORRY можно сохранить как канонические.

Добавлено поверх PATB:

- разделитель команд : наряду с историческим ;;
- необязательный THEN;
- ссылки на стек и регистры МК-61;
- вещественные математические и тригонометрические функции;
- определение текущей экранной геометрии через COLS и ROWS;
- очистка игрового экрана CLS, ожидание клавиши PAUSE и чтение одного нажатия через INPUT ();
- экранный редактор, меню, C6/USB-хранилище и локализация.

Строки и номера

Строка начинается с десятичного номера:

```
10 PRINT "HELLO"  
20 END
```

Номер строки:

- целое десятичное число;
- диапазон 1...32767;
- хранится как целый адрес строки, а не как вещественное число;
- отделяется от команд одним или несколькими пробелами.

Рекомендуемый шаг нумерации - 10:

```
10 INPUT "A", A  
20 INPUT "B", B  
30 PRINT A+B  
40 END
```

Так между строками остается место для вставки:

```
10 INPUT A  
15 IF A<0 PRINT "NEG"  
20 PRINT A
```

Порядок выполнения программы определяется числовым порядком строк. Исходный текст желательно хранить и показывать в отсортированном виде. Если в исходнике встречаются две строки с одинаковым номером, компилятор считает это ошибкой WHAT?, а не молча выбрать одну из них.

Пустые строки в исходнике игнорируются.

Команды в строке

В одной строке может быть несколько команд:

```
10 A=1;B=2;PRINT A+B
```

Исторический разделитель — ;. Также принимается ::

```
10 A=1:B=2:PRINT A+B
```

Последними в строке должны быть только команды, которые безусловно передают управление или останавливают программу:

- GOTO;
- RETURN;
- END;
- STOP.

GOSUB и FOR могут продолжаться в той же строке, как в PAB:

```
10 GOSUB 100;PRINT "BACK"  
20 FOR I=1 TO 3;PRINT I;NEXT I
```

Например:

```
10 IF A>0 GOTO 100  
20 PRINT "A<=0"
```

А так писать не нужно:

```
10 GOTO 100;PRINT "UNREACHABLE"
```

Редактор программы

Редактор TinyBASIC должен быть построчным и использовать общий модуль редактора. Поведение окна, курсора, SMS-ввода и навигации должно совпадать с FOCAL.

Редактор показывает окно листинга вокруг текущей строки:

- на LCD1602 и других двухстрочных дисплеях - 2 строки программы;
- на графическом дисплее - 4, 5, 7 или 8 строк программы, в зависимости от настройки Строк.

Текущая строка отмечается маркером > или инверсией:

```
10 INPUT A  
>20 PRINT A*A  
30 END
```

На двухстрочном дисплее показываются текущая строка и соседняя строка:

```
>20 PRINT A*A  
30 END
```

Если строка длиннее ширины дисплея, строка прокручивается горизонтально вокруг текстового курсора. Номер строки желательно оставлять видимым, пока это возможно.

Навигация в редакторе

Навигация такая же, как в FOCAL:

Действие	Клавиша
Курсор на символ влево	стрелка влево
Курсор на символ вправо	стрелка вправо

Действие	Клавиша
Предыдущая строка программы	ШГ<-
Следующая строка программы	ШГ->
Принять строку и перейти к следующей	OK / Enter
Сохранить программу и выйти	MENU/ESC
Удалить символ слева (забой)	Cx
Очистить текущую строку	F + Cx (CF)
Удалить пустую текущую строку	F + Cx (CF) на пустой строке

OK в TinyBASIC-редакторе вставляет новую строку. Двоеточие : как разделитель команд вводится отдельной комбинацией, чтобы не конфликтовать с переходом на следующую строку.

Комбинация F + стрелка влево редактирующего действия не выполняет. В поле имени программы действуют те же однострочные правила: Cx удаляет один символ слева, а F + Cx очищает имя целиком. Клавиша Г забоем не является. Здесь Cx — красная клавиша очистки, а не цифровая клавиша 0; комбинация F + 0 по-прежнему вводит !. Ввод символов в имени использует ту же схему, что основной редактор: цифры вводятся без префикса, K + цифровая клавиша включает SMS-ввод, а ПП вводит пробел.

Ввод операторов

В отличие от FOCAL, TinyBASIC не должен вставлять операторы по одной клавише. Клавиши, которые в FOCAL вставляют ASK, BRANCH, PRINT, SET и другие операторы, в TinyBASIC не должны делать этого.

Команды вводятся как обычный текст:

```
PRINT
INPUT
GOTO
GOSUB
RETURN
FOR
NEXT
CLS
PAUSE
END
```

Чтобы набор оставался коротким, нужно использовать сокращения Tiny BASIC:

Полная форма	Краткая форма
PRINT	P.
INPUT	IN.
IF	I.
GOTO	G.
GOSUB	GOS.
RETURN	R.
FOR	F.
NEXT	N.
CLS	C.
PAUSE	PAU.
END	E.
REM	REM
LET	можно опустить
THEN	можно опустить

Например:

```
10 P. "HELLO"  
20 E.
```

эквивалентно:

```
10 PRINT "HELLO"  
20 END
```

Аббревиатура должна заканчиваться точкой, кроме тех слов, где краткая форма совпадает с полным словом (REM, NEW в direct-режиме, если он будет добавлен позже).

Ввод символов

TinyBASIC использует тот же общий принцип префиксов, что FOCAL:

- обычные 0...9 вводят цифры;
- K + SMS-клавиша включает SMS-ввод латинских букв или пробела;
- F + цифровая клавиша вводит спецсимвол;
- K включает ввод дополнительного символа на одно следующее нажатие;
- после ввода одного символа префикс сбрасывается.

SMS-ввод букв:

Клавиша	Цикл символов
K + 7	пробел
K + 8	A, B, C
K + 9	D, E, F
K + 4	G, H, I
K + 5	J, K, L
K + 6	M, N, O
K + 1	P, Q, R, S
K + 2	T, U, V
K + 3	W, X, Y, Z

После K + буквенной SMS-клавиши повтор той же цифры уже без K заменяет последнюю введенную букву на следующую из цикла. Другая буквенная SMS-клавиша начинает следующую букву. В SMS-режиме курсор показывается иначе, как _.

Из SMS-режима выводят:

- пауза ввода;
- пробел ПП, он же вводит пробел;
- клавиша 7, она завершает SMS-режим и вводит пробел;
- клавиша 0, она только завершает SMS-режим.

Цифровая K + 0 не начинает SMS-ввод.

Спецсимволы:

Клавиша	Символ
F + 0	!
F + 1	@

Клавиша	Символ
F + 2	#
F + 3	\$
F + 4	%
F + 5	^
F + 6	&
F + 7	*
F + 8	(
F + 9	)

Основные символы TinyBASIC:

Символ	Как нажать
пробел	ПП или SMS-клавиша 7
цифры	цифровые клавиши
.	клавиша десятичной точки
+, -, *, /	соответствующие арифметические клавиши
=	К + +
<	К + ШГ<-
>	К + -
(	К + стрелка влево или F + 8
)	К + стрелка вправо или F + 9
"	К + X<->Y
'	К + .
,	К + ПП
:	К + ОК
;	К + В/О
^	клавиша ВП или F + 5
!	К + С/П или F + 0

Эта таблица сочетаний К одинакова в редакторах TinyBASIC и FOCAL.

Составные операторы сравнения набираются из отдельных символов:

```
<>
<=
>=
```

Локализация

Текущий язык интерфейса влияет на меню, подсказки, сообщения редактора, ввод числа и ошибки. В русском режиме пользователь должен видеть русские сообщения.

Исходный текст программы при этом остается латинским и стабильным:

```
10 INPUT "A", A
20 PRINT SIN(A)
30 END
```

Не должно быть русских вариантов операторов или функций. Например, в русском режиме пользователь вводит PRINT, INPUT, GOTO, SIN, RND, а не русские ключевые слова.

Условная компиляция

TinyBASIC должен отключаться на этапе сборки. Флаг:

```
#ifndef MK61_ENABLE_TINYBASIC
#define MK61_ENABLE_TINYBASIC 1
#endif
```

Значение по умолчанию - 1, то есть TinyBASIC включен.

Если MK61_ENABLE_TINYBASIC равен 0, из прошивки должны исключаться:

- пункт меню TinyBASIC;
- редактор TinyBASIC;
- парсер и компилятор TinyBASIC;
- runtime TinyBASIC;
- хранилище программ TinyBASIC;
- тестовые или сервисные входы, которые доступны только для TinyBASIC.

Публичные функции TinyBASIC в заголовке должны иметь пустые inline-заглушки, чтобы остальной код мог собираться без дополнительных `#ifdef` в местах вызова.

Общий редактор не должен иметь отдельного пользовательского флага включения. Он является библиотекой: если включен FOCAL или TinyBASIC, язык подключает редактор; если оба языка отключены, редактор не должен попадать в прошивку.

Хранение программ

TinyBASIC-программа должна храниться отдельно от обычных программ МК-61 и FOCAL.

Формат хранения:

```
type/signature = B2
extension      = TBI
name
source_length
source_text
checksum
```

Сигнатура B2 отделяет TinyBASIC от остальных типов хранилища, чтобы файл другого языка нельзя было случайно загрузить как TinyBASIC-программу.

Расширение virtual FAT:

```
NAME.TBI
```

Практичная схема работы:

- при открытии редактора программа загружается из постоянного хранилища в RAM;
- редактор работает с RAM-копией;
- при выходе из редактора программа компилируется или проверяется;
- если проверка прошла, исходник записывается обратно в слот;
- при запуске сохраненная программа читается из слота и компилируется в рабочее представление;
- если места не хватает, пользователь получает ошибку SORRY / нет места.

Компилированное или токенизированное представление можно держать в RAM как кэш. Сохраненный текст должен оставаться источником истины. Так программу проще редактировать, переносить через terminal/USB и обновлять между версиями прошивки.

Текущие пределы:

Ресурс	Предел
Исходный текст	3584 байта
Строки программы	192
Вложенные GOSUB	16
Активные FOR	16
Глубина выражения	96
Переменные	26 (A–Z)

Текст вместе с заголовком и именем помещается в один сектор данных С6.

Direct-команды

Исторический Tiny BASIC имел direct-команды RUN, LIST, LIST n и NEW. В MK61s mini основным интерфейсом является меню, поэтому отдельной текстовой direct-консоли нет. Её действия представлены интерфейсом устройства.

Соответствие меню:

Историческая команда	Действие в MK61s mini
RUN	пункт меню Запуск
LIST	редактор программы
LIST n	навигация к строке в редакторе
NEW	пункт Новая в выборе программ

В сохранённой программе direct-команды не используются.

Операторы

Канонические операторы:

```
REM
LET
PRINT
INPUT
IF
GOTO
GOSUB
RETURN
FOR
NEXT
CLS
PAUSE
END
STOP
```

Краткие формы с точкой должны приниматься парсером:

```
P. "HELLO"
IN. A
I. A>0 P. A
G. 100
GOS. 200
R.
F. I=1 TO 10
N. I
E.
```

LET можно опускать:

```
10 LET A=10
20 B=A*2
```

THEN можно опускать:

```
10 IF A>0 THEN PRINT A
20 IF A>0 PRINT A
```

Обе строки выше эквивалентны.

REM

REM начинает комментарий.

```
10 REM SUM OF SQUARES
20 A=3:B=4
30 PRINT A^2+B^2
```

Комментарий продолжается до конца строки. Двоеточия внутри комментария не разделяют команды:

```
10 REM THIS : IS COMMENT
```

REMARK можно принять как полную форму, но каноническая форма для набора на калькуляторе - REM.

LET

LET присваивает значение переменной:

```
10 LET A=234-5*6
20 LET B=A/2
```

LET можно опустить:

```
10 A=234-5*6
20 B=A/2
```

Последовательное присваивание из Tiny BASIC сохраняется:

```
10 LET A=1,2,3
```

Эта строка присваивает:

```
A=1
B=2
C=3
```

Последовательное присваивание начинается с указанной переменной и идет по алфавиту. Если список выходит за Z, это ошибка NOW?.

Историческая явная форма списка также поддерживается:

```
10 LET A=4,B=5,@(0)=A+B
```

Целью присваивания может быть переменная, элемент @() или ссылка МК-61.

PRINT

PRINT выводит значения и строки:

```
10 PRINT "HELLO"
20 PRINT A
30 PRINT A*3+1, "ABC", 'CBA'
```

Строки можно заключать в двойные или одинарные кавычки:

```
10 PRINT "TEXT"
20 PRINT 'TEXT'
```

Пустой PRINT выводит перевод строки:

```
10 PRINT
```

Разделитель , переводит вывод к следующей табуляционной позиции. Разделитель ; печатает следующий элемент сразу после предыдущего:

Табуляционные позиции расположены через 8 символов (0, 8, 16, ...).

```
10 PRINT "A=", A
20 PRINT "A="; A
```

Если список PRINT заканчивается , или ;, перевод строки после команды не добавляется:

```
10 PRINT "A=";
20 PRINT A
```

#n устанавливает ширину следующих числовых полей (0...63):

```
10 PRINT #8,A,B
```

В оригинальном PATB начальная ширина поля была 8. На коротком экране МК61s mini по умолчанию используется компактная ширина 0; исторический вид получается явным PRINT #8, . . .

^A...^Z выводят соответствующий управляющий ASCII-символ.

INPUT

INPUT читает выражения в переменные, элементы массива или ссылки МК-61:

```
10 INPUT A
20 INPUT A,B
30 INPUT @(0),.R0
```

Перед вводом по умолчанию показывается короткая подсказка:

:

Строки в списке INPUT заменяют подсказку:

```
10 INPUT "A", A
20 INPUT "B", B
```

Как в PATB, запятую после строки-подсказки можно опустить:

```
10 INPUT "A?"A
```

Можно вводить несколько переменных:

```
10 INPUT "A", A, "B", B
```

При вводе принимается полное числовое выражение, например 2*PI или A+1. В исходном тексте и вводимом выражении десятичная точка пишется как ..

Cx удаляет один символ слева, F + Cx очищает всё введённое число. Клавиша Г забоем не является.

ESC во время числового ввода останавливает программу, не записывая частично набранное значение.

Стек и регистры МК-61

TinyBASIC может читать и писать стек и регистры калькулятора через ссылки с точкой. Стековые ссылки:

```
.X .Y .Z .T
```

Регистры памяти:

```
.R0 .R1 .R2 .R3 .R4 .R5 .R6 .R7
.R8 .R9 .RA .RB .RC .RD .RE
```

В расширенном режиме памяти также доступна ссылка .RF. В обычном режиме памяти .RF считается ошибкой.

Ссылки можно использовать в выражениях:

```
10 A=.X+1
20 PRINT .R0
```

Запись выполняется обычным присваиванием:

```
10 .X=42
20 LET .R0=.X+1
30 A=.R0
```

INPUT может записывать число прямо в стек или регистр:

```
10 INPUT .Y
20 A=.Y
```

Последовательное присваивание LET A=1, 2, 3 работает только для обычных переменных A - Z. Для ссылок МК-61 справа должно быть одно выражение:

```
10 .R0=1
```

В FOR ссылку МК-61 использовать нельзя. Управляющая переменная цикла остается обычной переменной TinyBASIC:

```
10 FOR I=1 TO 10
20 .R0=I
30 NEXT I
```

IF

IF проверяет выражение и выполняет оставшуюся часть строки, если выражение ненулевое:

```
10 IF A>B PRINT "A"
20 IF A=B THEN PRINT "EQ"
```

Если выражение равно нулю, оставшаяся часть строки пропускается, а выполнение продолжается со следующей строкой.

После условия может идти одна команда или несколько команд через ; или :.

```
10 IF A<0 PRINT "NEG":A=-A
20 PRINT A
```

ELSE не поддерживается. Ветвление с альтернативой пишется через GOTO:

```
10 IF A<0 GOTO 100
20 PRINT "POS"
30 GOTO 200
100 PRINT "NEG"
200 END
```

GOTO

GOTO передает управление строке:

```
10 GOTO 100
```

Адрес может быть выражением:

```
10 GOTO A*10+B
```

Результат выражения должен быть целым номером строки. Если результат не является целым номером или такой строки нет, выполнение останавливается с ошибкой NOW?.

GOTO должен быть последней командой в строке.

GOSUB и RETURN

GOSUB вызывает подпрограмму:

```
10 GOSUB 100
20 END
100 PRINT "SUB"
110 RETURN
```

RETURN возвращает выполнение к команде после последнего GOSUB.

GOSUB может быть вложенным. Глубина стека вызовов должна быть фиксированной и проверяться. Если стек переполнен или RETURN выполнен без GOSUB, это ошибка NOW?.

После возврата выполнение продолжается непосредственно за вызовом, в том числе в той же строке:

```
10 GOSUB 100;PRINT "BACK"
```

Максимальная глубина GOSUB — 16. RETURN завершает текущую строку.

FOR и NEXT

FOR начинает цикл:

```
10 FOR I=1 TO 10
20 PRINT I
30 NEXT I
```

Семантика:

- начальное выражение вычисляется один раз;
- конечное выражение после TO вычисляется один раз при входе в цикл;
- STEP вычисляется один раз; по умолчанию шаг равен 1;
- положительный, отрицательный и нулевой шаги соответствуют PATB;
- начальное значение вне диапазона сразу передаёт управление за парный NEXT;
- NEXT I прибавляет сохранённый шаг и сравнивает с сохранённым пределом.

FOR, тело и NEXT могут находиться в одной строке:

```
10 FOR I=5 TO 1 STEP -2;PRINT I;NEXT I
```

NEXT ищет активный FOR с указанной переменной и удаляет более свежие несовпадающие циклы, как исторический интерпретатор. Новый FOR с уже активной управляющей переменной заменяет прежний. Максимум одновременно активны 16 циклов.

END и STOP

END завершает выполнение программы:

```
10 PRINT "DONE"
20 END
```

STOP делает то же самое:

```
10 STOP
```

END можно не писать в последней строке программы. Если выполнение дошло до конца программы, это считается нормальным завершением.

END и STOP должны быть последними командами в строке.

CLS и PAUSE

CLS очищает экран и возвращает следующую строку PRINT в левый верхний угол. PAUSE ждёт нажатия любой физической клавиши. ESC во время PAUSE завершает программу; остальные клавиши продолжают выполнение. Если после последнего PAUSE программа не меняла экран и завершилась, среда запуска не требует второго, скрытого подтверждения:

```
10 CLS
20 PRINT "YOUR MOVE"
30 PAUSE
```

Обе команды принимают обычные сокращения с точкой (C., PAU.) и не имеют аргументов.

INPUT(): одно нажатие клавиши

INPUT () ждёт одно новое нажатие и возвращает числовой код клавиши. Ввод подтверждать ОК не требуется. Экран, шрифт и позиция вывода сохраняются; подсказку или игровое поле программа рисует сама через PRINT.

```
10 PRINT "LEFT / RIGHT / OK"
20 K=INPUT ( )
30 IF K=15 X=X-1
40 IF K=16 X=X+1
50 IF K=19 END
60 GOTO 20
```

INPUT () не имеет аргументов; скобки обязательны. Допустимо сокращение INP . (). Результат можно записать в переменную, массив или регистр, например @(0)=INPUT () или .R0=INPUT ().

Каждый вызов читает отдельное нажатие: INPUT ()+INPUT () ждёт две клавиши. Проверка и компиляция исходника клавиш не читают; вызов в невыполненной ветви IF также не ждёт ввода. Отпускание и повтор от удержания не считаются новым нажатием. Удержанная клавиша должна быть отпущена перед следующим нажатием.

Логические коды совпадают с публичным API APP и не зависят от раскладки Mini, Classic или 40TH:

Клавиша	Код
0...9	0...9
десятичная точка	10
+, -, ×, ÷	11, 12, 13, 14
←, →	15, 16
ШГ←, ШГ→	17, 18
ОК	19
С/П	21
Сх	22
К, F	23, 24
USER	25
ПП, БП	26, 27
хП, Пх	28, 29
В/О	30
дополнительные клавиши шага APP	31, 32

Клавиши с несколькими назначениями возвращают первый соответствующий логический код, как в API APP. Для остальных физических клавиш возвращается 256 + scan-code; эти дополнительные коды зависят от раскладки.

ESC, как в PAUSE, завершает запуск. Код 20 в переменную не записывается. Прерванное выражение не присваивает результат; последующие вызовы INPUT () в нём не выполняются. В M61-сценарии отмена передаётся наружу без сообщения поверх игрового экрана. Если после INPUT () программа завершается без нового вывода, дополнительного подтверждения не требуется.

Функция используется в выражениях программы. Выражение, набираемое внутри обычного INPUT, не может содержать INPUT () и запускать вложенное ожидание клавиши.

Переменные

Есть 26 числовых переменных:

A B C ... Z

Переменные хранят калькуляторные числа. Начальное значение всех переменных - 0.

Значения сохраняются между последовательными запусками TinyBASIC. Пункт меню Clear DATA / Сброс данных возвращает все переменные и массив к 0.

Числовой массив PATB адресуется выражением:

```
10 @ (I)=I*I
20 PRINT @ (I)
```

Допустимые индексы — целые числа от 0 до $\text{INT}(\text{SIZE}/2)$ включительно. SIZE сообщает совместимый объём свободной области; фактический максимум ограничен 8-КиБ языковым workspace и уменьшается с ростом текста программы.

Строковых переменных и пользовательских функций нет. Строки используются как литералы в PRINT, INPUT и REM.

Выражения

Поддерживаются:

- калькуляторные числа с десятичной точкой;
- переменные A...Z;
- элементы массива @(expr) и значение SIZE;
- PI;
- круглые скобки;
- арифметика +, -, *, /, ^, MOD;
- унарные +, -, NOT;
- сравнения >, <, =, #, <>, >=, <=;
- логические операции AND, OR, XOR;
- вызовы встроенных функций.

Примеры:

```
A+B
2*PI*R
SIN(A)^2+COS(A)^2
1/(A+B)
SQR(A^2+B^2)
(A>B)*100+(A<=B)*200
```

Оператор ^ означает возведение в степень:

```
X^Y
```

MOD для калькуляторных чисел определяется так:

```
A MOD B = A - INT(A/B)*B
```

Деление или MOD на ноль - ошибка NOW?.

Приоритет операций

Базовый порядок вычисления:

- 1 круглые скобки и вызовы функций;
- 2 возведение в степень ^;
- 3 унарные операции +, -, NOT;
- 4 *, /, MOD, AND;
- 5 +, -, OR, XOR;
- 6 сравнения >, <, =, <>, >=, <=.

Внутри одного уровня операции вычисляются слева направо, кроме случаев, которые будут явно зафиксированы тестами для калькуляторной совместимости.

Сравнения возвращают 1 или 0.

Логические операции в TinyBASIC булевы:

- 0 считается ложью;
- любое ненулевое значение считается истиной;
- результат AND, OR, XOR, NOT равен 1 или 0.

Это отличается от PATB, где логические операции были побитовыми над 16-битными целыми числами.

Функции МК-61

Математические функции TinyBASIC используют тот же канонический набор, что FOCAL:

SIN (X)	COS (X)	TG (X)
ASIN (X)	ACOS (X)	ATG (X)
LN (X)	LG (X)	EXP (X)
SQRT (X)	ABS (X)	
INT (X)	FRAC (X)	ROUND (X)
SGN (X)	MAX (X, Y)	
RND ()		
RND (X)		
INPUT ()		
PI	SIZE	
COLS	ROWS	

Исторические функции A. (ABS), R. (RND) и S. (SIZE) принимаются. Расширенные функции также можно сокращать точкой; из-за конфликта с SIZE синус сокращается как SIN. , а не SI. .

ROUND (X) округляет половины от нуля: ROUND (1.5) дает 2, а ROUND (-1.5) дает -2.

RND () возвращает результат калькуляторной операции СЧ.

RND (X) - совместимость с Tiny BASIC. Она возвращает случайное целое от 1 до INT (X) включительно. Если X < 1, это ошибка HOW?. Внутри RND (X) используется тот же калькуляторный генератор, что и RND ().

COLS и ROWS возвращают число столбцов и строк текущего текстового поля. Это значения без аргументов и без скобок; имена намеренно не сокращаются. Если TinyBASIC запущен из M61-сценария с временным шрифтом, геометрия отражает этот шрифт. Так программа может адаптировать разметку или явно отказаться от запуска на слишком маленьком экране:

```
10 IF COLS<40 GOTO 100
```

Сам TinyBASIC шрифт не выбирает. Он только использует экранную геометрию, которую подготовила запускающая его оболочка. Благодаря этому отдельные части игры могут запускаться последовательно, не повторяя экранную настройку в каждой BASIC-программе.

При запуске из M61-сценария экран сохраняется между BASIC-частями, включая части, которые выполняют только вычисления. Для явной очистки используйте CLS. Обычный запуск из меню TinyBASIC начинается с чистого экрана.

Файл autoexes. tbi, лежащий непосредственно в папке, запускается при входе в неё из Проводника. Если рядом есть autoexes. m61, приоритет остаётся у него.

Алиасы не поддерживаются:

- нет TAN, только TG;
- нет ATAN, только ATG;
- нет LOG, только LG;
- нет SQR, вместо этого выражение X^2;
- нет INV, вместо этого выражение 1/X;

- нет EXP10, вместо этого выражение 10^X .

Тригонометрический режим

Тригонометрические функции должны использовать текущий тригонометрический режим калькулятора, если это технически возможно.

```
10 INPUT "A", A
20 PRINT SIN(A)
30 END
```

Поведение SIN, COS, TG, ASIN, ACOS, ATG должно совпадать с калькуляторным поведением МК-61 и тестироваться отдельно.

Редакторские макросы выражений

TinyBASIC не использует FOCAL-операторные макросы и не вставляет операторы по одной клавише.

Калькуляторные макросы выражений (x^2 , $1/x$, 10^x) можно добавить позже, если они не будут конфликтовать с обычным вводом TinyBASIC. Базовая версия должна оставаться простой: все команды и функции набираются текстом или сокращениями.

Примеры программ

Hello

```
10 PRINT "HELLO"
20 END
```

Короткая форма:

```
10 P. "HELLO"
20 E.
```

Сумма двух чисел

```
10 INPUT "A", A
20 INPUT "B", B
30 PRINT "S="; A+B
40 END
```

Квадраты от 1 до 10

```
10 FOR I=1 TO 10
20 PRINT I, I^2
30 NEXT I
40 END
```

Гипотенуза

```
10 INPUT "A", A
20 INPUT "B", B
30 PRINT SQR(A^2+B^2)
40 END
```

Модуль числа

```
10 INPUT A
20 IF A<0 A=-A
30 PRINT A
40 END
```

Через функцию:

```
10 INPUT A
20 PRINT ABS(A)
30 END
```

Подпрограмма

```
10 A=3
20 GOSUB 100
30 A=4
40 GOSUB 100
50 END
100 PRINT A^2
110 RETURN
```

Выбор строки выражением

```
10 INPUT "N", N
20 GOTO 100+10*N
100 PRINT "ZERO"
110 PRINT "ONE"
120 PRINT "TWO"
130 END
```

Такой прием требует, чтобы вычисленная строка существовала. Если строки нет, будет ошибка HOW?.

Кубик

Через калькуляторный RND():

```
10 D=1+INT(6*RND())
20 PRINT D
30 END
```

Через совместимый RND(X):

```
10 PRINT RND(6)
20 END
```

Ошибки

Канонические короткие ошибки Tiny BASIC:

```
WHAT?
HOW?
SORRY
```

Рекомендуемая семантика:

- WHAT? - синтаксическая ошибка: неизвестная команда, неправильная строка, незакрытая кавычка, неправильное выражение.
- HOW? - команда понятна, но выполнить ее нельзя: деление на ноль, несуществующая строка в GOTO, RETURN без GOSUB, переполнение стека, неправильная переменная в NEXT.
- SORRY - нехватка памяти или места в хранилище.

В русском интерфейсе можно показывать локализованные сообщения, но короткие формы стоит сохранить в отладке и тестах, потому что они компактны и исторически узнаваемы.

Что не входит

- Побитовая 16-битная семантика AND, OR, XOR, NOT.
- 16-битное целочисленное переполнение оригинального PATB.
- ELSE.
- DATA, READ, RESTORE.
- строковые переменные;
- пользовательские функции;
- отдельная текстовая direct-консоль;
- одно-клавишная вставка операторов в редакторе TinyBASIC.

NEXT без переменной не принимается. Короткие ошибки WHAT?, HOW? и SORRY остаются каноническими, а экранные сообщения локализуются.